

AGENDA

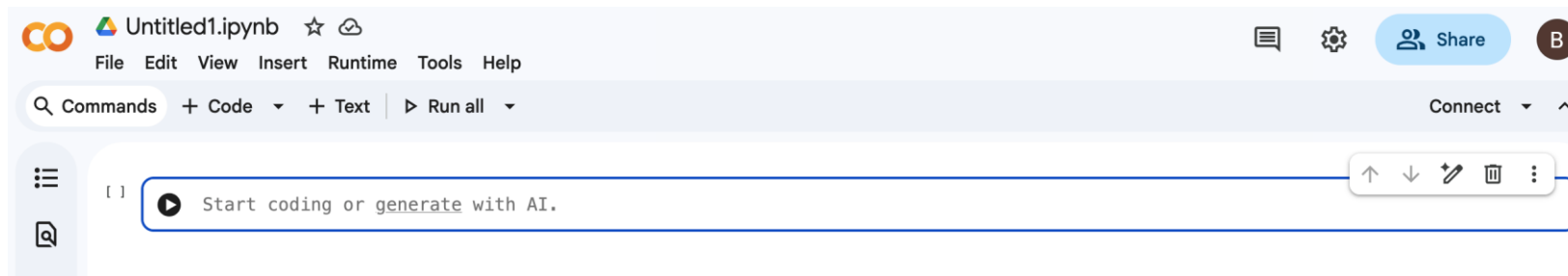
- Output, input, and variables
- Types
- Arithmetic operators, including `//` and `%`
- Operator precedence, and the `+=` shortcuts
- Comparison operators and booleans
- Converting between types
- What arithmetic operators do to strings
- `len()`, indexing, and off-by-one
- while loops
- for loops

a note on the textbook

- Most of the material in this talk is drawn from the first three chapters of the textbook
- The textbook is available here:
<https://allendowney.github.io/ThinkPython/>

development environment

- I will be using google colab for livecoding this semester
- You will *also* be using google colab for doing your assignments
- To get started, go to colab.new
- You will enter code in the box you see at the top of the screen
- To run code, click the play button at the left side of the box
- Note: to save, you will have to log into a google account



hello world: your first python program

- Behold: hello world!
- (by convention, the first program everyone writes in a language they're learning is a program to print out "Hello world!" and then exit)

```
1 print("Hello world!")
```

comments: notes for humans

- Anything after a # is a **comment**
- Python ignores it completely — it is there for people reading the code
- Use comments to explain *why* you did something, not *what* the line does
- On these slides I will use them to show you what each line prints

```
1  # this whole line is a comment
2  print("Hello world!")    # so is this part
3
4  # print("this line never runs")
```

```
$ python3 comments.py
Hello world!
```

creating variables

- You can store values for future use in *variables*
- To create a variable, give

- 1) the name of the variable
- 2) =
- 3) and the value you want to assign

```
1  magic_number = 137      # int
2  price = 4.99            # float
3  greeting = "Hello!"    # str
4  is_ready = True         # bool
```

printing a literal string vs a variable

- A quoted string prints exactly what's between the quotes

A variable name (*no quotes*) prints the value stored inside it

- Python decides which one you mean by whether you used quotes

```
1  name = "Ava"

3  print("name")    # prints: name
4  print(name)      # prints: Ava
```

variables have types

- When you store a value in a variable, it's assigned a *type*
- `int` — whole numbers: 137, 0, -4
- `float` — numbers with a decimal point: 3.5, 2.0
- `str` — text: "hello", "137"
- `bool` — True or False
- The type of a value decides what the operators do to it

```
1 magic_number = 137      # int
2 price = 4.99            # float
3 greeting = "Hello!"    # str
4 is_ready = True         # bool
```

Note: boolean values (bools) are True and False, with capital letters, not true and false

type(): ask Python what you are holding

- `type()` is a built-in function, like `print()` and `input()`
- It tells you what kind of value a variable holds
- Reach for it whenever a program is doing something you cannot explain
- Look closely at that last line

```
1 print(type(137))           # <class 'int'>
2 print(type(4.99))          # <class 'float'>
3 print(type("hello"))       # <class 'str'>
4 print(type("137"))          # <class 'str'>
```

arithmetic operators: the full set

- These are the arithmetic operators we have in Python
- `+`, `-`, `*`, and `/` are (as you'd expect) add, subtract, multiply, and divide
- There's a few more, though: `//`, `%`, and `**`
- We'll discuss them on the following slides

```
1  print(7 + 3)      # 10
2  print(7 - 3)      # 4
3  print(7 * 3)      # 21
4  print(7 / 3)      # 2.3333333333333335
5  print(7 // 3)     # 2
6  print(7 % 3)      # 1
7  print(7 ** 3)     # 343
```

** : raising to a power

- ** means *to the power of*
- `2 ** 10` is 2 multiplied by itself 10 times
- A fractional power gives you a root
- There is no ^ operator for powers in Python — ^ means something else entirely

```
1 print(2 ** 10)      # 1024
2 print(5 ** 2)       # 25
3 print(9 ** 0.5)     # 3.0    square root
4 print(2 ** -1)      # 0.5
```

/ always gives you a float

- In Python 3, / is *true division*
- It hands back a `float` even when the numbers divide evenly
- `10 / 5` is `2.0`, not `2`

```
1 print(10 / 5)      # 2.0      not 2
2 print(7 / 2)       # 3.5
3 print(1 / 3)       # 0.3333333333333333
4 print(9 / 3)       # 3.0      still a float
```

(What's a float? "float" means "floating-point number", which for our purposes is a number with a decimal component. We'll discuss this more later on in this slideset, when we discuss *types*)

// : whole-number division

- // throws the fractional part away
- Read it as *how many whole times does this fit?*
- Use it when a fraction of a thing makes no sense — people, pages, coins

```
1 print(7 // 2)      # 3
2 print(10 // 5)     # 2
3 print(1 // 3)      # 0
4 print(99 // 10)    # 9
```

// vs / side by side

- The same two numbers, two different answers
- / keeps the remainder as a decimal
- // discards it
- Neither one *rounds* — // always goes down

```
1  print(7 / 2)      # 3.5
2  print(7 // 2)     # 3
3
4  print(9 / 4)      # 2.25
5  print(9 // 4)     # 2
6
7  print(5 / 10)     # 0.5
8  print(5 // 10)    # 0
```

// gotcha: it floors, it does not truncate

- // rounds *down*, toward negative infinity
- For positive numbers, rounding down and chopping off the decimal are the same thing
- For negative numbers they are **not**
- `-7 // 2` is `-4`, not `-3`

```
1  print(7 // 2)      # 3
2  print(-7 // 2)     # -4    not -3
3
4  print(1 // 10)     # 0
5  print(-1 // 10)    # -1    not 0
```

// gotcha: floats in, float out

- // gives a whole-number *value*, but not always an `int` *type*
- If either side is a `float`, the answer is a `float`
- The `.0` on the end is your clue

```
1 print(7 // 2)           # 3
2 print(7.0 // 2)         # 3.0
3 print(7 // 2.0)         # 3.0
4 print(7.5 // 2)         # 3.0
```


% : the remainder

- % gives you what is left over after division
- `17 % 5` is 2, because 5 fits into 17 three times with 2 left over
- `//` gives the quotient, % gives the remainder
- Together they tell you everything about a division

```
1  print(17 // 5)      # 3    how many times
2  print(17 % 5)       # 2    what is left over
3
4  print(20 % 5)       # 0    divides evenly
5  print(3 % 5)        # 3    5 never fits
```

% in practice: odd or even

- A number is even when dividing by 2 leaves nothing over
- So $n \% 2$ is 0 for even numbers and 1 for odd ones
- This is the standard way to test even and odd, in every language

```
1 print(10 % 2)      # 0 -> even
2 print(11 % 2)      # 1 -> odd
3 print(12 % 2)      # 0 -> even
4 print(13 % 2)      # 1 -> odd
```

% in practice: wrapping around

- % keeps a number inside a fixed range
- Anything % 12 lands between 0 and 11
- This is how clocks, calendars and card decks work

```
1  # 9 o'clock, plus 5 hours
2  print((9 + 5) % 12)      # 2
3
4  print(25 % 12)           # 1
5  print(100 % 12)          # 4
```

// and % together

- Between them they take a number apart
- // gives you the big units, % gives you the leftovers
- 137 seconds is 2 minutes and 17 seconds

```
1 total_seconds = 137
2
3 minutes = total_seconds // 60
4 seconds = total_seconds % 60
5
6 print(minutes, "minutes and", seconds, "seconds")
```

```
$ python3 split-time.py
2 minutes and 17 seconds
```

dividing by zero

- Python refuses, and stops the program with a `ZeroDivisionError`
- This is a *runtime* error — the program starts fine, then crashes partway through
- All three of `/`, `//` and `%` do this

```
1  print("about to divide")
2  print(10 / 0)
3  print("we never get here")
```

```
$ python3 oops.py
about to divide
Traceback (most recent call last):
  File "oops.py", line 2, in <module>
    print(10 / 0)
ZeroDivisionError: division by zero
```

which operation happens first?

- Python does not simply work left to right
- `**` first, then `*` `/` `//` `%`, then `+` `-`
- The same precedence rules you learned in algebra
- `2 + 3 * 4` is 14, not 20

```
1 print(2 + 3 * 4)      # 14    not 20
2 print(10 - 2 ** 3)    # 2
3 print(1 + 8 // 2)     # 5
4 print(20 - 6 % 4)     # 18
```

parentheses win

- Anything inside parentheses happens first
- Use them even when you do not strictly need them
- Code that is obvious to read beats code that is clever

```
1  print(2 + 3 * 4)          # 14
2  print((2 + 3) * 4)        # 20
3
4  # the average of three scores
5  print((80 + 90 + 100) / 3) # 90.0
```

a shortcut: +=

- Adding something to a variable is so common it has its own operator
- `counter += 1` means exactly `counter = counter + 1`
- Read it as *add this to what is already there*

```
1 counter = 0
2
3 counter = counter + 1    # the long way
4 counter += 1            # the short way
5
6 print(counter)          # 2
```


`+=` has a whole family

- Every arithmetic operator has a matching shortcut

```
1  score = 100
2
3  score += 10      # 110
4  score -= 30      # 80
5  score *= 2       # 160
6  score /= 3       # 53
7  score %= 50      # 3
8  score **= 2      # 9
```

comparing two values

- A comparison asks a question about two values
- == equal, != not equal
- < > <= >= for ordering

```
1 print(5 == 5)      # True
2 print(5 != 5)      # False
3 print(3 < 5)        # True
4 print(3 > 5)        # False
5 print(5 <= 5)       # True
6 print(5 >= 6)       # False
```

= is not ==

- = **assigns**: it puts a value into a variable
- == **asks**: are these two things equal?
- Confusing them is the most common beginner bug there is
- Say them out loud differently: *gets* versus *equals*

```
1 age = 20           # assignment: age gets 20
2
3 print(age == 20)    # a question: True
4 print(age == 21)    # a question: False
5 print(age)          # still 20
```

comparisons give you booleans

- A comparison evaluates to True or False
- That is a type of its own, called bool
- True and False are capitalised, and they are not strings
- This is exactly what a while loop needs after the while keyword

```
1  result = 7 > 3
2  print(result)           # True
3
4  user_input = "quit"
5  print(user_input != "quit") # False
```

checkpoint: predict the output

- Work each line out on paper **before** you run it
- Write your answer down — guessing in your head does not count
- Then type them in and see which ones you missed

```
1 print(9 // 4)
2 print(9 % 4)
3 print(-9 // 4)
4 print(20 % 6)
5 print(10 - 2 * 3)
6 print((10 - 2) * 3)
7 print(7 // 2 == 3)
```

GETTING INPUT

Here's a short program that asks the user for a word and then tells them what they type.

(not the most useful program in the world, but...)

```
1 word = input("Please enter a word: ")
2
3 print("You typed:", word)
```

INPUT()

Here's a short program that asks the user for a word and then tells them what they type.
(not the most useful program in the world, but...)

Just like you use **print()** to send text to the screen, you use **input()** to read text from the keyboard.

`input()` returns whatever the user typed, so you usually assign it straight to a variable, like `word = input(...)`.

```
1 word = input("Please enter a word: ")
2
3 print("You typed:", word)
```

input()

Just like you use **print()** to send text to the screen, you use **input()** to read text from the keyboard.

Whatever the user types gets returned by `input()`, so you assign it to a variable to save it.

```
1 word = input("Please enter a word: ")
2
3 print("You typed:", word)
```

Notice there are no quotes around `word` in the `print()` call. This tells Python I don't want to print the *word* "word" — I want to print the contents of the variable named **word**

input() always hands you a string

- Whatever the user types, `input()` gives it back as a `str`
- Even when they typed nothing but digits
- `"137"` and `137` are **not** the same thing
- This is the single most important fact about `input()`

```
1 age = input("How old are you? ")
2
3 print(age)           # 20
4 print(type(age))     # <class 'str'>
```

```
$ python3 age.py
How old are you? 20
20
<class 'str'>
```

the bug this causes

- Here is a program that should add two numbers together
- It does not crash
- It just quietly gives the wrong answer
- Those are the worst bugs to find

```
1 first = input("First number: ")
2 second = input("Second number: ")
3
4 print("The sum is:", first + second)
```

```
$ python3 add.py
First number: 5
Second number: 3
The sum is: 53
```

why 53?

- Both values were strings, so + did what + does to strings
- It glued them together instead of adding them
- Python did exactly what you asked — the *values* were the wrong type
- The operator is innocent here

```
1  print(5 + 3)           # 8      two ints
2  print("5" + "3")       # 53     two strs
3
4  print(type(5))          # <class 'int'>
5  print(type("5"))        # <class 'str'>
```

int(): turning a string into a number

- `int()` converts a value into an integer
- Wrap it around `input()` and you get a number you can do arithmetic on
- This is called a **type conversion**, or a cast

```
1 age = int(input("How old are you? "))
2
3 print(age)           # 20
4 print(type(age))     # <class 'int'>
5 print(age + 1)       # 21
```

the fixed program

- One change on each `input()` line
- Now both values are `ints`, so `+` adds them
- Convert as early as you can — right where the value comes in

```
1 first = int(input("First number: "))
2 second = int(input("Second number: "))
3
4 print("The sum is:", first + second)
```

```
$ python3 add.py
First number: 5
Second number: 3
The sum is: 8
```

float() and str()

- `float()` gives you a number that can have a decimal point
- Use it when the user might reasonably type 2.5
- `str()` goes the other way, turning a number back into text

```
1 price = float(input("Price: "))
2 print(price * 2)           # 5.0
3
4 count = 42
5 print(str(count) + " items") # 42 items
```

int() is picky

- `int()` only works when the string really looks like a whole number
- Anything else stops the program with a `ValueError`
- The message names the exact value it choked on — read it

```
1 age = int(input("How old are you? "))
2 print("Next year you will be", age + 1)
```

```
$ python3 age.py
How old are you? twenty
Traceback (most recent call last):
  File "age.py", line 1, in <module>
    age = int(input("How old are you? "))
ValueError: invalid literal for int() with base 10: 'twenty'
```

type errors: reading the message

- A `TypeError` means you used an operator on types it does not accept
- Read the **last** line first — it says what went wrong
- Then read the line number above it to find where
- The message here is unusually helpful: it names both types

```
1 count = 42
2 print("You have " + count + " items")
```

```
$ python3 items.py
Traceback (most recent call last):
  File "items.py", line 2, in <module>
    print("You have " + count + " items")
TypeError: can only concatenate str (not "int") to str
```


strings: + glues them together

- On strings, + means **concatenation**
- It joins them end to end, with nothing in between
- Notice that you have to supply your own spaces

```
1 first = "Ada"
2 last = "Lovelace"
3
4 print(first + last)           # AdaLovelace
5 print(first + " " + last)    # Ada Lovelace
```

+ needs strings on both sides

- Python will not quietly turn a number into text for you
- Either convert the number yourself with `str()`
- Or hand both to `print()` with a comma and let it do the work

```
1 count = 42
2
3 # print("You have " + count)      TypeError
4
5 print("You have " + str(count))    # You have 42
6 print("You have", count)           # You have 42
```

strings: * repeats them

- `str * int` repeats the string that many times
- Useful for separator lines and simple formatting
- `str * str` is a `TypeError` — repeating something a *string* number of times is meaningless

```
1 print("ha" * 3)           # hahaha
2 print("-" * 20)          # -----
3 print("=" * 5 + ">")      # =====>
4
5 # print("ha" * "3")       TypeError
```

what you cannot do to strings

- Only + and * are defined for text
- - and / have no sensible meaning on strings, so Python refuses
- There is no obvious answer to "hello" / 2, so Python does not invent one

```
1 print("hello" + "world")      # helloworld
2 print("hello" * 3)            # hellohellohello
3
4 # print("hello" - "lo")       TypeError
5 # print("hello" / 2)          TypeError
```

print(a, b) is not print(a + b)

- A comma tells print() to display several things, with a space between them
- + builds one single string before print() ever sees it
- The comma also handles numbers for you; + does not
- When in doubt, use the comma

```
1 count = 42
2
3 print("You have", count, "items")
4 print("You have " + str(count) + " items")
5
6 print("a", "b")      # a b      space added
7 print("a" + "b")     # ab       no space
```

`+=` works on strings too

- The same shortcut applies
- `message += "!"` means `message = message + "!"`
- This is how you build a string up one piece at a time

```
1 message = "Hello"
2
3 message += ", world"
4 message += "!"
5
6 print(message)      # Hello, world!
```

len(): how long is it?

- len() is a built-in function that counts the characters in a string
- Spaces and punctuation count too
- The empty string "" has length 0

```
1 print(len("hello"))           # 5
2 print(len("hello!"))          # 6
3 print(len("two words"))       # 9
4 print(len(""))                # 0
```

len() with a variable

- `len()` measures what is *inside* the variable, not the name of it
- Combine it with `input()` to measure whatever the user typed
- `len()` hands you back an `int`, so you can do arithmetic with it

```
1 word = input("Enter a word: ")
2
3 print("You typed:", word)
4 print("That is", len(word), "characters")
```

```
$ python3 length.py
Enter a word: python
You typed: python
That is 6 characters
```


reaching one character: []

- Square brackets pull a single character out of a string
- The number inside is called the **index**
- `word[0]` is the first character

```
1 word = "python"
2
3 print(word[0])      # p
4 print(word[1])      # y
5 print(word[2])      # t
```

counting starts at zero

- The first character sits at index 0, not 1
- So a six-character string uses indexes 0 through 5
- This trips up nearly everyone at first — it is worth drawing out

```
1 word = "python"
2 #      012345
3
4 print(word[0])      # p      first
5 print(word[5])      # n      last
6 print(len(word))    # 6
```

the last character is at $\text{len}(s) - 1$

- The length is always one more than the largest valid index
- So the last character sits at $\text{len}(\text{word}) - 1$
- That $- 1$ is the famous **off-by-one**, and it will bite you

```
1 word = "python"
2
3 last_index = len(word) - 1
4
5 print(last_index)           # 5
6 print(word[last_index])    # n
```

IndexError: off the end

- Asking for an index that does not exist stops the program
- `word[6]` on a six-character string is one step too far
- When you see `IndexError`, go looking for a missing `- 1`

```
1 word = "python"  
2 print(word[6])
```

```
$ python3 index.py  
Traceback (most recent call last):  
  File "index.py", line 2, in <module>  
    print(word[6])  
IndexError: string index out of range
```

counting from the end

- A negative index counts backwards from the end of the string
- `word[-1]` is the last character, with no arithmetic needed
- `word[-2]` is the one before it
- Both spellings of *the last character* are shown here

```
1 word = "python"
2
3 print(word[-1])      # n
4 print(word[-2])      # o
5
6 print(word[len(word) - 1])  # n
```

len() + range(): visiting every character

- `range(len(word))` produces every valid index for that string
- For a six-letter word that is 0, 1, 2, 3, 4, 5
- Which is exactly what a for loop wants
- That is where we are going next

```
1 word = "python"
2
3 for i in range(len(word)):
4     print(i, word[i])
```

```
$ python3 spell.py
0 p
1 y
2 t
3 h
4 o
5 n
```

checkpoint: predict the output

- Same rules — write your answer down before you run anything
- Two of these are errors
- Say **which** error each one is, and why

```
1 print("5" + "3")
2 print(int("5") + int("3"))
3 print("ab" * 3)
4 print(len("hello world"))
5 print("hello"[1])
6 print("hello"[len("hello")])
7 print("3" * "4")
```

review: while

- while loops repeat *while* a given boolean expression is true
- The boolean expression that controls the while loop goes right after the while keyword.
- This loop repeats until the user types "quit."

```
1 user_input = "NONE"
2 counter = 0
3
4 while user_input != "quit":
5     user_input = input("Type quit to quit: ")
6     counter += 1
7
8 print("The counter is:", counter)
```


review: while

- while loops repeat *while* a given boolean expression is true
- The boolean expression that controls the while loop goes right after the while keyword.
- NOTE: Python only checks if a loop is done after each statement in the loop has been evaluated – so even when the user types "quit," the counter is still incremented.

```
1 user_input = "NONE"
2 counter = 0
3
4 while user_input != "quit":
5     user_input = input("Type quit to quit: ")
6     counter += 1
7
8 print("The counter is:", counter)
```

loop condition becomes false on line 5...

review: while

- while loops repeat *while* a given boolean expression is true
- The boolean expression that controls the while loop goes right after the while keyword.
- NOTE: Python only checks if a loop is done after each statement in the loop has been evaluated – so even when the user types "quit," the counter is still incremented.

```
1 user_input = "NONE"
2 counter = 0
3
4 while user_input != "quit":
5     user_input = input("Type quit to quit: ")
6     counter += 1
7
8 print("The counter is:", counter)
```

... but line 6 is still executed...

review: while

- while loops repeat *while* a given boolean expression is true
- The boolean expression that controls the while loop goes right after the while keyword.
- NOTE: Python only checks if a loop is done after each statement in the loop has been evaluated – so even when the user types "quit," the counter is still incremented.

```
1 user_input = "NONE"
2 counter = 0
3
4 while user_input != "quit":
5     user_input = input("Type quit to quit: ")
6     counter += 1
7
8 print("The counter is:", counter)
```

... and only THEN do we check if the loop condition is false

FOR LOOPS

- for loops repeat *for* a certain number of steps
- The syntax uses `range()` to say how many times to repeat, plus a loop variable that takes on each value in turn — no semicolons needed

```
1  for counter in range(10):  
2      print("the counter is at...", counter)
```

```
$ python3 first-for.py  
the counter is at... 0  
the counter is at... 1  
the counter is at... 2  
the counter is at... 3  
the counter is at... 4  
the counter is at... 5  
the counter is at... 6  
the counter is at... 7  
the counter is at... 8  
the counter is at... 9
```

FOR LOOPS

- for loops repeat *for* a certain number of steps
- The syntax uses `range()` plus a loop variable — no semicolons needed
- `range(10)` generates the numbers 0 through 9. The loop variable, *counter*, takes on each of those values in turn, starting at 0.

```
1  for counter in range(10):  
2      print("the counter is at...", counter)
```

```
$ python3 first-for.py  
the counter is at... 0  
the counter is at... 1  
the counter is at... 2  
the counter is at... 3  
the counter is at... 4  
the counter is at... 5  
the counter is at... 6  
the counter is at... 7  
the counter is at... 8  
the counter is at... 9
```

FOR LOOPS

- for loops repeat *for* a certain number of steps
- The syntax uses `range()` plus a loop variable — no semicolons needed
- There's no separate *boolean condition* to write at all — the loop simply exits once `range()` runs out of numbers

```
1  for counter in range(10):  
2      print("the counter is at...", counter)
```

```
$ python3 first-for.py  
the counter is at... 0  
the counter is at... 1  
the counter is at... 2  
the counter is at... 3  
the counter is at... 4  
the counter is at... 5  
the counter is at... 6  
the counter is at... 7  
the counter is at... 8  
the counter is at... 9
```

for loops

- for loops repeat *for* a certain number of steps
- The syntax uses `range()` plus a loop variable — no semicolons needed
- There's no separate *increment* step either — `range()` advances counter automatically each time through the loop.

```
1  for counter in range(10):  
2      print("the counter is at...", counter)
```

```
$ python3 first-for.py  
the counter is at... 0  
the counter is at... 1  
the counter is at... 2  
the counter is at... 3  
the counter is at... 4  
the counter is at... 5  
the counter is at... 6  
the counter is at... 7  
the counter is at... 8  
the counter is at... 9
```

FOR vs. while

- Typically we use `for` loops when we know how many times a loop will repeat, and `while` loops when we *don't* know how many times we'll repeat (for example, when we'll repeat until the user types a certain value).
- However, everything you can do with a `for` loop, you could do with a `while` loop instead.